

Risoluzione del problema Gioco del 15

Andrea Ciccarello

Gennaio 2025

Contents

1	Introduzione	1
2	Fluenti e azioni	2
3	Modello e Soluzione	3
3.1	Parametri e costanti	3
3.2	Stati e fluenti	3
3.3	Definizione di adiacenza e azione <code>muovi_spazio</code>	3
3.4	Possibilità e scelta dell'azione	4
3.5	Transizione di stato	4
3.6	Stato iniziale e goal	4
3.7	Vincoli e ottimizzazione	4
3.8	Geometria del taxi	5
4	Risultati e Benchmark	5
5	Configurazione per la risoluzione	8
6	Tool aggiuntivi	10

1 Introduzione

Sliding puzzle è un noto problema di pianificazione in cui una serie di tessere numerate devono essere ordinate su una griglia, muovendole una posizione alla volta in uno spazio vuoto. Tale problema rappresenta una sfida sia dal punto di vista della complessità computazionale che della strategia di risoluzione, rendendolo un caso di studio ideale per metodologie basate sulla logica.

L'approccio tradizionale per risolvere lo sliding puzzle si basa su algoritmi di ricerca dello stato, quali A*, che richiedono una definizione esplicita delle regole di transizione e delle euristiche. In alternativa, l'uso di linguaggi dichiarativi come ASP permette di rappresentare il problema attraverso vincoli logici, concentrandosi sulla descrizione delle relazioni tra gli stati e delle condizioni obiettivo piuttosto che sui dettagli procedurali.

2 Fluenti e azioni

La rappresentazione all'interno del programma sfrutta i fluenti e le azioni per poter rappresentare lo stato della griglia.

Un *fluente* esprime una proprietà di un oggetto del mondo. Tramite i fluenti vengono descritte le proprietà salienti della realtà modellata. Solitamente il valore (logico) di un fluente dipende dal tempo. Le azioni, quando eseguite, hanno l'effetto di modificare lo stato del mondo, ovvero, modificando l'insieme dei fluenti veri, portano il mondo da uno stato ad un altro.

Una *situazione* è la rappresentazione della sequenza delle azioni compiute a partire da uno stato iniziale del mondo. Rappresentando la situazione priva di azioni (quella iniziale) con la lista vuota, allora la scrittura $[a_n, \dots, a_1]$ denota la situazione ottenuta compiendo le azioni $a_1, \dots, a_n$, nell'ordine, a partire dallo stato iniziale.

Per descrivere la transizione da uno stato al successivo, per effetto dell'esecuzione di una azione si può utilizzare una *effect proposition* della forma:

$$a \text{ causes } f \text{ if } p_1, \dots, p_n, \neg q_1, \dots, \neg q_r$$

dove a è una azione, f un *fluent literal* e i p_i e i q_j sono fluenti. Questa proposizione asserisce che se i fluenti $p_1, \dots, p_n, \neg q_1, \dots, \neg q_r$ sono veri in uno stato σ , o meglio: in una situazione che corrisponde ad uno stato σ , allora estendere tale situazione con l'esecuzione dell'azione a porta ad uno stato in cui sarà vero il fluente f .

Oltre a descrivere la evoluzione delle situazioni bisogna sfruttare una notazione per descrivere delle osservazioni sul mondo modellato. Ovvero un mezzo per asserire che (o chiedere se) una certa proprietà sia o meno vera in uno stato/situazione. Questo viene fatto tramite *initially*.

Per esempio all'interno del problema le configurazioni inizializzate vengono rappresentate nella forma

```
1 initially(posizione_tessera(10, 3, 2)).
2 initially(posizione_spazio(3, 4)).
3 initially(posizione_tessera(13, 4, 1)).
4 ...
```

deve il *posizione_tessera* rappresenta dove nella griglia è stata posizionata ogni tessera. Lo spazio vuoto viene rappresentato con *posizione_spazio*.

Similmente tramite goal si riesce a rappresentare lo stato finale della griglia

```
1 goal(posizione_tessera(1, 1, 1)).
2 goal(posizione_tessera(2, 1, 2)).
3 ...
```

Per poter rappresentare griglie più grandi a livello di goal si inseriscono più termini, arrivando a 15 posizioni totali. Per le diverse configurazioni della griglia sono stati prodotti i rispettivi stati iniziali e goal.

L'immagine rappresenta la griglia 3x3 nella la configurazione di goal.

Tempo 14

1	2	3
4	5	6
7	8	

Figure 1: Configurazione finale del gioco del 8

3 Modello e Soluzione

Il gioco del 15 prevede di spostare una delle possibili 4 celle mobili al posto dello spazio. Se lo spazio si trova adiacente ad un bordo le celle di riducono a 3 (2 nel caso di uno spazio nell'angolo). Inizialmente il problema è stato affrontato, partendo dalla risoluzione del gioco del 8, versione semplificata rispetto al puzzle del 15 perchè si ha una griglia 3×3 al cui interno possono essere inserite 8 tessere in totale. Successivamente si è passati alla versione 3×4 e 4×4 . I risultati saranno presentati per tutte e 3 le configurazioni.

Nel problema, si considera una griglia di dimensioni $nr \times nc$, che contiene tessere numerate da 1 a $(nr \times nc - 1)$ e una casella vuota (lo *spazio*).

3.1 Parametri e costanti

Vengono definite le costanti nr , nc e $maxtime$, che rappresentano rispettivamente il numero di righe, il numero di colonne e il tempo massimo di ricerca. Vengono quindi introdotti i domini:

$$domR = \{1, \dots, nr\}, \quad domC = \{1, \dots, nc\}, \quad time = \{0, \dots, maxtime\}.$$

All'interno di questa griglia, ogni posizione (X, Y) con $X \in domR$ e $Y \in domC$ può contenere una tessera oppure lo spazio vuoto.

3.2 Stati e fluenti

Uno *stato* al tempo T ($T \in time$) è specificato dai fluenti:

$$holds(posizione_spazio(X, Y), T) \quad holds(posizione_tessera(Tid, X, Y), T)$$

dove Tid è un identificativo di tessera. Il primo fluente indica che lo spazio vuoto si trova in (X, Y) all'istante T , mentre il secondo indica che la tessera Tid è nella posizione (X, Y) all'istante T .

3.3 Definizione di adiacenza e azione muovi_spazio

Le posizioni (X_1, Y_1) e (X_2, Y_2) sono dette *adiacenti* se differiscono di una sola unità in riga oppure in colonna, senza uscire dai domini $domR$ e $domC$.

3.4 Possibilità e scelta dell'azione

Un'azione di movimento $muovi_spazio(X_1, Y_1, X_2, Y_2)$ è possibile al tempo T se:

$$\begin{aligned} &possibile(muovi_spazio(X_1, Y_1, X_2, Y_2), T) \rightarrow \\ &\quad holds(posizione_spazio(X_1, Y_1), T) \wedge \\ &\quad holds(posizione_tessera(-, X_2, Y_2), T) \wedge \\ &\quad adiacente(X_1, Y_1, X_2, Y_2). \end{aligned}$$

La selezione dell'azione in ciascun istante avviene in modo non deterministico tramite la regola:

$$\{ occurs(muovi_spazio(X_1, Y_1, X_2, Y_2), T) \} \rightarrow possibile(muovi_spazio(X_1, Y_1, X_2, Y_2), T).$$

3.5 Transizione di stato

Se l'azione $muovi_spazio(X_1, Y_1, X_2, Y_2)$ accade al tempo T , allora al tempo $T + 1$:

$$holds(posizione_spazio(X_2, Y_2), T + 1),$$

e la tessera che si trovava in (X_2, Y_2) si sposta in (X_1, Y_1) :

$$holds(posizione_tessera(Tid, X_1, Y_1), T + 1).$$

Ogni altro fluente $posizione_spazio(\cdot)$ o $posizione_tessera(\cdot)$ persiste inalterato da T a $T + 1$, a meno che un'azione di movimento o il raggiungimento del goal non lo modifichino. In questo modello è possibile eseguire soltanto 1 azione per istante temporale.

3.6 Stato iniziale e goal

La configurazione iniziale è determinata dai fatti $initially(\cdot)$, validi al tempo 0. Il *goal* è raggiunto in un tempo T se:

$$(\forall Tid, X, Y, goal(posizione_tessera(Tid, X, Y)) \rightarrow holds(posizione_tessera(Tid, X, Y), T))$$

e, se specificato, anche

$$goal(posizione_spazio(Gx, Gy)) \rightarrow holds(posizione_spazio(Gx, Gy), T).$$

Non si generano ulteriori stati se il goal è già stato raggiunto.

3.7 Vincoli e ottimizzazione

Tra i vincoli più importanti figurano:

- Il divieto di eseguire due azioni distinte nel medesimo istante.
- Il divieto di compiere azioni dopo il raggiungimento del *goal*.
- Il taglio dei rami che presentano degli evidenti loop

Per l'ottimizzazione, si introduce la funzione obiettivo:

$$\textit{minimize}\{ T : \textit{goal_reached}(T) \},$$

che seleziona il modello con il tempo di raggiungimento del goal *minimo*.

L'output del grounder va da un minimo di 35k righe nel caso 3x3 ad un massimo di 100k righe nel caso 4x4. Gran parte del tempo viene utilizzato per eseguire il solver tramite clasp.

3.8 Geometria del taxi

la geometria dei taxi è una metrica che permette di calcolare la distanza di 1 tessera dalla sua posizione di goal, in questo modo è possibile indicare al ASP-solver quali siano le mosse preferibili.

$$\mathbf{p} = (p_1, p_2, \dots, p_n) \quad \text{e} \quad \mathbf{q} = (q_1, q_2, \dots, q_n)$$

è definita da

$$d_{\text{Manhattan}}(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n |p_i - q_i|.$$

La Distanza di Manhattan, sotto forma di vincoli debole, non ha diminuito in modo significativo i tempi di esecuzione.

4 Risultati e Benchmark

I risultati ottenuti sono stati realizzati tramite 100 benchmark per configurazione della griglia (3x3, 3x4 e 4x4), in totale sono quindi stati eseguiti 300 benchmark. I benchmark sono stati creati tramite uno script che ha permesso di produrre configurazioni iniziali per il puzzle.

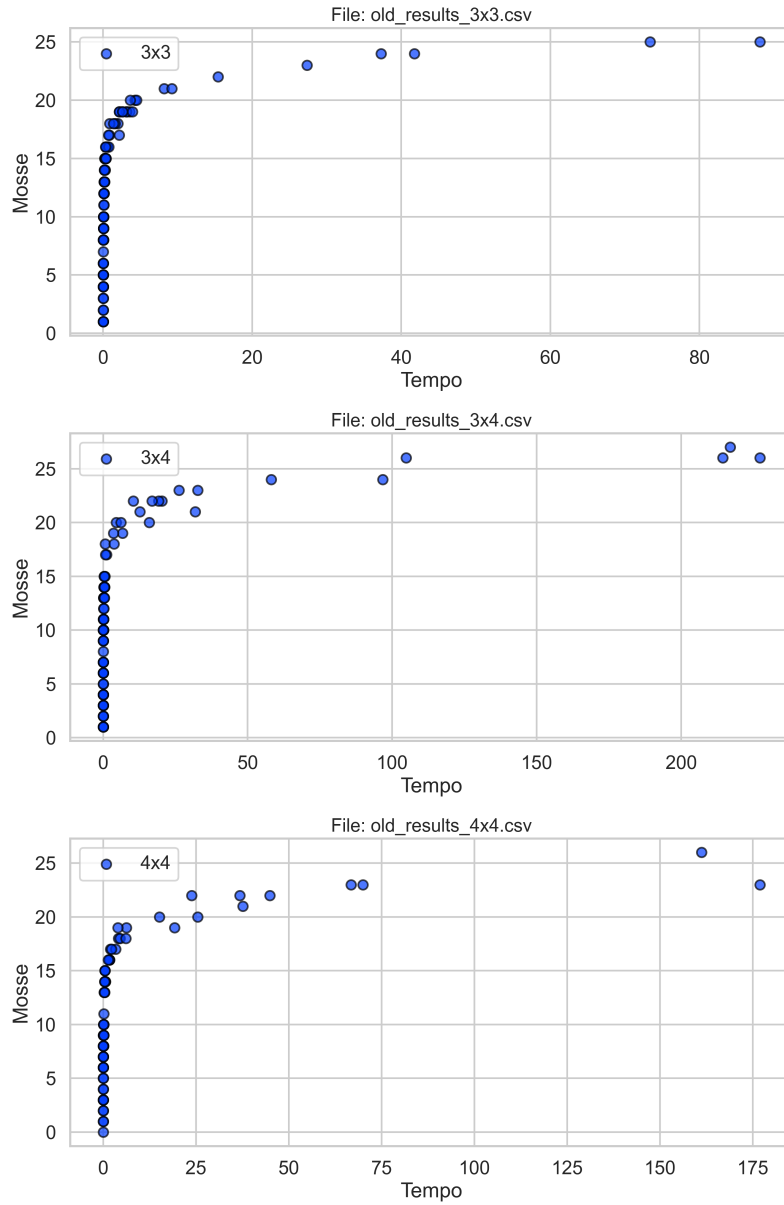


Figure 2: Benchmark risultati nelle 3 configurazioni

La seconda visualizzazione offre una migliore visione d'insieme, sfruttando la visualizzazione della scala logaritmica per l'asse del tempo. è evidente come la crescita dei tempi sia esponenziale man mano che le configurazioni iniziali hanno un maggiore numero di mosse minime per essere risolte.

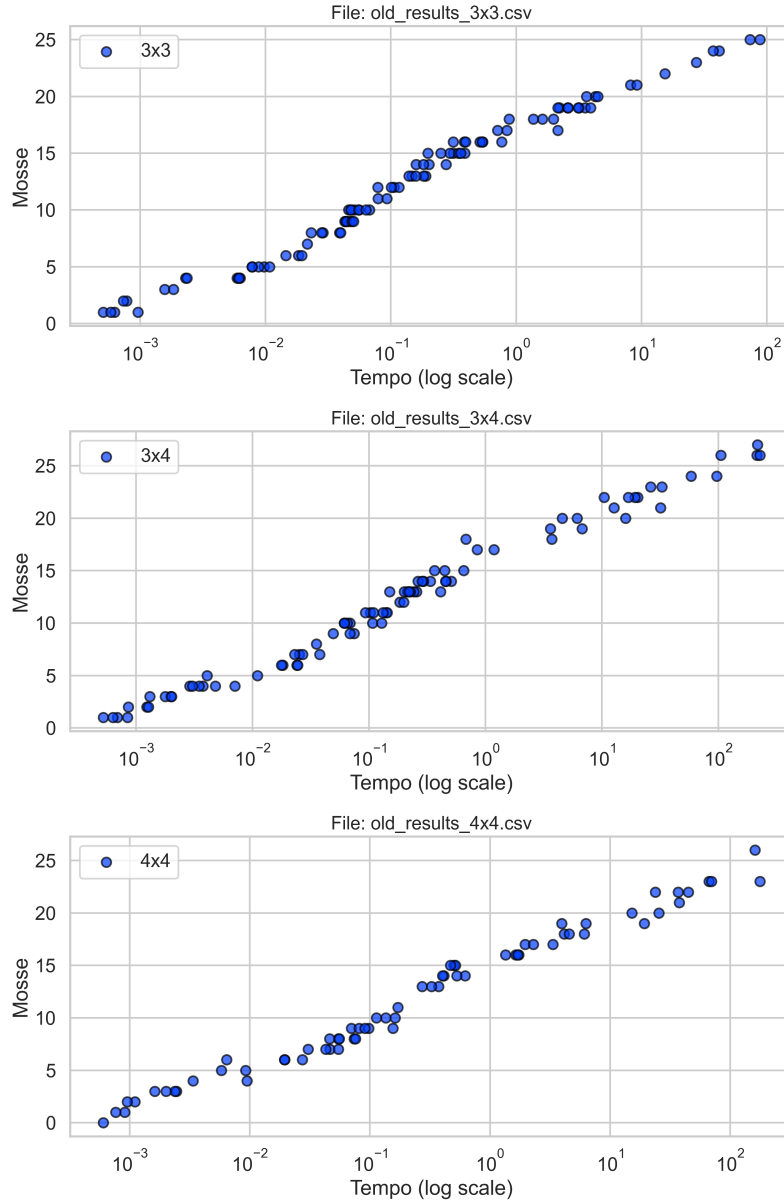


Figure 3: Benchmark scala log nei tempi

Il limite della calcolabilità con 300 secondi e 8 thread si raggiunge con una distanza di 28 mosse dalla configurazione di goal. I risultati oltre i 300 secondi non sono stati mostrati nei plot. Per questi test non è stata sfruttato il calcolo della distanza, tramite la geometria del taxi, per guidare il solver dato che in questo esempio non ho constatato evidenti differenze.

5 Configurazione per la risoluzione

I test per la migliore configurazione del solver sono stati fatti su una configurazione 3x4 risolvibile con un minimo di 17 mosse. Tale configurazione è stata scelta perché il tempo impiegato per la sua risoluzione senza alcun parametro risulta di 200 secondi di CPU Time. I benchmark tengono conto del CPU time effettivo dato da clasp. In prima analisi sono andato ad analizzare le macro configurazioni messe a disposizione da clasp tramite il flap `-configuration`:

`auto` Seleziona la configurazione in base al tipo di problema.

`frumpy` Utilizza impostazioni predefinite conservative.

`jumpy` Utilizza impostazioni predefinite aggressive.

`tweety` Utilizza impostazioni predefinite orientate ai problemi ASP.

`handy` Utilizza impostazioni predefinite orientate ai problemi di grandi dimensioni.

`crafty` Utilizza impostazioni predefinite orientate ai problemi creati su misura.

`trendy` Utilizza impostazioni predefinite orientate ai problemi industriali.

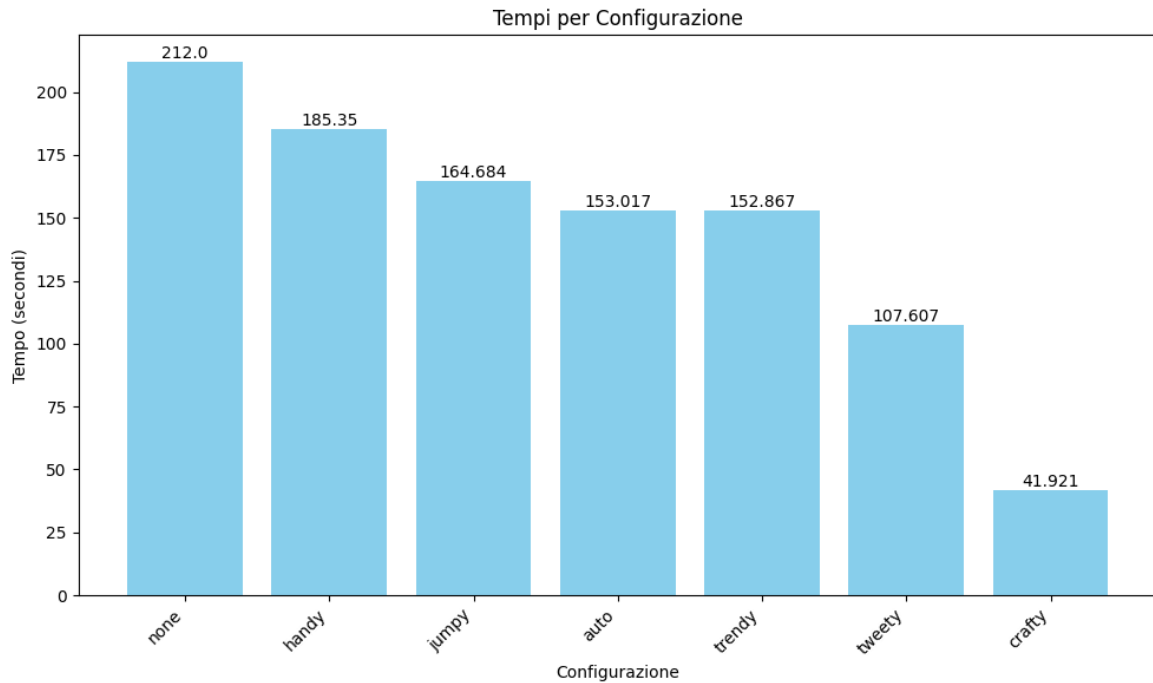


Figure 4: Tempi per Configurazione

La migliore configurazione si è dimostrata essere *Crafty* in questo caso. Tale configurazione già utilizza l'euristica Vsids (Chaff-like). Successivamente sono passato all'analisi delle ottimizzazioni dove ho notato il maggior speedup grazie all'ottimizzazione basata su nuclei insoddisfacibili minimi.

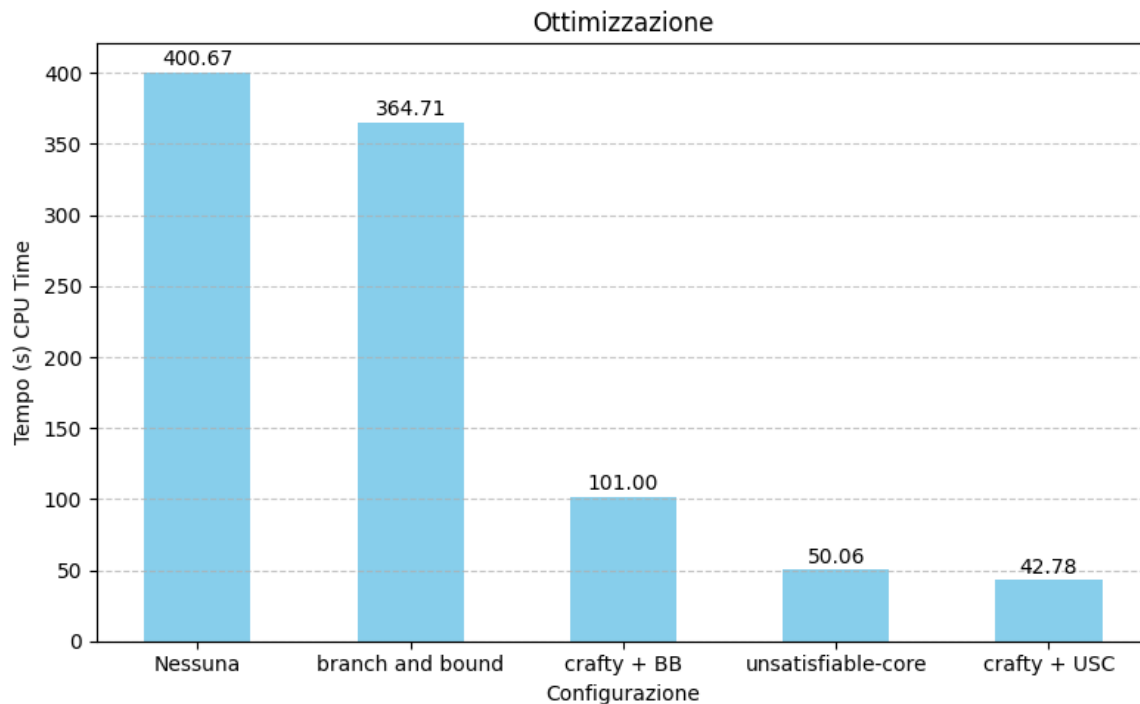


Figure 5: Tempi per Configurazione

Un nucleo insoddisfacibile (unsatisfiable core) è un sottoinsieme di clausole di una formula logica che è insoddisfacibile. Questa strategia di ottimizzazione è particolarmente efficace in questo problema perché in quanto si sta cercando il minimo numero di mosse per poter risolvere una data combinazione di tessere iniziali, quindi se si deduce che sia impossibile trovare una soluzione entro un certo limite inferiore allora si possono scartare le soluzioni sotto a tale bound. Il solver sta creando delle assunzioni secondo cui esista una soluzione sotto una certo numero di mosse, se tale assunzione è falsa allora può alzare il limite inferiore del problema.

```
Reading from gioco_generico.asp ...
Solving...
Progression : [4;inf]
Progression : [5;inf]
Progression : [6;inf]
...
Progression : [13;inf]
Progression : [14;inf]
Progression : [15;inf]
```

Normalmente il solver troverebbe delle soluzioni vicino al limite superiore imposto tramite *maxtime*. Clasp mette a disposizione anche la possibilità di ottenere una prima fase di preprocessing del modello prima di partire con la soluzione tramite USC, tale fase però non ha diminuito i tempi di esecuzione.

Dato l'overhead dovuto all'esecuzione parallela ho deciso di utilizzare un esempio più complesso per vedere se la configurazione Crafty permetta di migliori i tempi caso dell'algoritmo di ottimizzazione del nucleo insoddisfacibile.

- Crafty con USC $\rightarrow$ 791 secondi
- Soltanto USC $\rightarrow$ 1037 secondi

Come strategia per effettuare i benchmark ho utilizzato una versione iterativa del problema che sfruttasse come configurazione Crafty.

6 Tool aggiuntivi

All'interno del progetto sono stati sviluppati diversi tool aggiuntivi pensati per visualizzare il risultato della computazione. In particolare, ho sviluppato un visualizzatore che permette di creare la visualizzazione della griglia in ogni istante temporale e, da essa, produrre un video o una GIF animata.

Tempo 0			Tempo 15		
4	1	2	1	2	3
8	6	7	4	5	6
5		3	7	8	

(a) Posizione iniziale a tempo T=0

(b) Goal ottenuto dal modello a tempo T=15

Figure 6: Visualizzazioni a differenti istanti temporali

Tale tool sfrutta i predicati *hold* che vengono generati dal Answare set. Per eseguire i test ho richiamato il modello, le posizioni iniziali e il goal da uno script python esterno